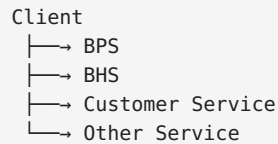


API Gateway — Spring Cloud Gateway

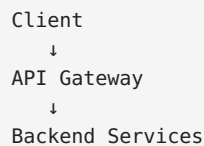
1. What is an API Gateway?

An **API Gateway** is a **single entry point** for clients calling **backend services**.

Instead of clients directly knowing every backend service:



they can call:



The gateway can handle common concerns such as:

- routing
- authentication/authorization integration
- rate limiting
- CORS
- logging
- request/response filtering
- load balancing/service discovery integration
- resilience policies

2. Spring Cloud Gateway

Spring Cloud Gateway is Spring's gateway implementation for Spring-based microservices.

A request comes in:



It is built around a **reactive/non-blocking architecture**, using Spring WebFlux.

3. Simple Example

Suppose the client calls:

```
GET /api/payments/123
```

Gateway can route it to:

```
payment-service
```

Conceptually:

```
/api/payments/**  
↓  
BPS
```

A route configuration can look like:

```
spring:  
  cloud:  
    gateway:  
      routes:  
        - id: bps-route  
          uri: http://localhost:8080  
          predicates:  
            - Path=/api/payments/**
```

So:

```
Client  
↓  
/api/payments/123  
↓  
Gateway  
↓  
BPS
```

4. What is a Route?

A Gateway route tells it:

■ **Which incoming request should go to which backend service?**

A route generally consists of:

```
Route  
├─ ID  
├─ URI  
├─ Predicates  
└─ Filters
```

Example:

```
routes:  
  - id: bps
```

```
uri: http://bps-service
predicates:
  - Path=/api/payments/**
```

5. Predicates

A **predicate** determines whether a route matches the incoming request.

Examples include:

```
Path
Method
Header
Host
Query
Cookie
```

Example:

```
predicates:
  - Path=/api/payments/**
  - Method=GET,POST
```

Meaning:

```
Path matches /api/payments/**
AND
HTTP method is GET or POST
```

6. Filters

Filters modify or process the request/response before it reaches the backend or before it returns to the client.

Examples:

```
AddRequestHeader
RemoveRequestHeader
RewritePath
AddResponseHeader
```

Conceptually:

```
Client
↓
Gateway
↓
Request Filter
↓
BPS
↓
Response Filter
↓
Client
```

7. Why use an API Gateway?

Without gateway:

```
BASAPI → BPS
Majesco → BPS
Client → BHS
Client → Other Service
```

Each consumer may need to understand:

- service locations
- authentication
- API versions
- network details
- policies

With gateway:

```
BASAPI
  ↓
Gateway
  ↓
BPS

Majesco
  ↓
Gateway
  ↓
BPS
```

Common policies can be centralized.

8. API Gateway vs Load Balancer

They are related but different.

Load Balancer

Primarily distributes traffic among instances:

```
BPS
├─ Instance 1
├─ Instance 2
└─ Instance 3
```

API Gateway

Provides an application-level entry point and can handle:

```
Routing
Authentication
Filtering
Rate limiting
CORS
```

A gateway may also participate in load balancing.

9. API Gateway vs Service Discovery

This is a **very common interview question**.

Service Discovery

Answers:

Where are the available instances of BPS?

```
Service Registry
  ↓
BPS instances
```

API Gateway

Answers:

Where should this incoming request be routed, and which common policies should be applied?

```
Client
  ↓
Gateway
  ↓
BPS
```

They can work together:

```
Client
  ↓
Gateway
  ↓
Service Discovery
  ↓
BPS instance
```

10. Gateway + Eureka

If the architecture uses Eureka:

```
      Eureka
      ↑
BPS-1 —|
BPS-2 —|
BPS-3 —|

Client
  ↓
Gateway
```

```
↓  
Eureka / Load Balancer  
↓  
BPS instance
```

Gateway doesn't have to hard-code:

```
http://10.0.1.10:8080
```

It can use a logical service name.

For example, depending on the configuration:

```
uri: lb://BPS
```

The `lb://` scheme indicates load-balanced service resolution.

11. Gateway Filters in a Real Application

Suppose BASAPI sends:

```
POST /payments  
Authorization: Bearer ...  
X-Correlation-ID: ABC123
```

Gateway might:

1. Receive request
2. Validate/forward authentication information
3. Add correlation ID if missing
4. Apply routing rules
5. Forward to BPS
6. Capture response metadata
7. Return response

BPS then handles the actual payment business logic.

Important:

■ The gateway should generally not contain your core payment business logic.

For example, don't put:

```
"Is this payment already processed?"  
"Should Hydra or Chase process it?"
```

into the gateway simply because it is convenient.

That belongs to the appropriate business service/domain layer.

12. Authentication at Gateway

A gateway can be used to centralize authentication-related processing.

For example:

```
Request
↓
Gateway
↓
Token validation
↓
BPS
```

But exactly where authorization is enforced depends on the architecture.

Do not assume:

“Gateway handles security, so backend services don’t need authorization.”

Backend services should still enforce appropriate trust boundaries and authorization.

13. Rate Limiting

Suppose BASAPI suddenly sends:

```
10,000 requests/sec
```

Gateway can apply rate limiting:

```
Incoming traffic
↓
Gateway
↓
Rate limiter
↙   ↘
allowed rejected
↓
BPS
```

This protects downstream services from excessive traffic.

For payment systems, rate limiting can help protect infrastructure, but shouldn’t be confused with fraud controls or payment authorization.

14. CORS at Gateway

Because you asked about CORS earlier, this is a useful connection.

Gateway can be configured to handle CORS:

```
Browser
↓
Gateway
↓
BPS
```

The gateway may return headers such as:

```
Access-Control-Allow-Origin
Access-Control-Allow-Methods
Access-Control-Allow-Headers
```

But whether CORS belongs at the gateway, backend, or both depends on the architecture.

15. Path Rewriting

Suppose the public API is:

```
/api/v1/payments/123
```

but BPS internally expects:

```
/payments/123
```

Gateway can rewrite the path.

Conceptually:

```
/api/v1/payments/123
  ↓
Gateway
  ↓
/payments/123
  ↓
BPS
```

Spring Cloud Gateway provides filters such as `RewritePath` for this type of transformation.

16. Timeouts

Gateway should have sensible connection/response timeout settings.

Example:

```
Client
  ↓
Gateway
  ↓ timeout
BPS
```

But don't use excessively aggressive timeouts for a payment system.

A timeout is also not the same as a payment failure.

For example:

```
Gateway
  ↓
BPS
  ↓
Hydra
  ↓
```

```
Payment processed
↓
Response delayed/lost
```

The gateway seeing a timeout doesn't necessarily mean the payment didn't happen.

This is a very important payment-system distinction.

17. Gateway and Retry

You should be cautious here.

For a read-only request:

```
GET /payments/123
```

automatic retries may often be safer.

For:

```
POST /payments
```

blind retries can be dangerous.

Example:

```
POST payment
↓
Gateway retries
↓
BPS
↓
Hydra processes payment twice
```

Therefore payment APIs need **idempotency**, not simply "retry everything."

18. Circuit Breaker

Gateway can also participate in resilience mechanisms.

Conceptually:

```
Client
↓
Gateway
↓
BPS
↓
Downstream
```

If BPS/downstream repeatedly fails:

```
CLOSED
↓
Failures
```

```
↓
OPEN
↓
Stop sending requests temporarily
```

Spring Cloud Gateway can integrate with resilience mechanisms such as Spring Cloud CircuitBreaker depending on the architecture.

19. Gateway vs BPS in Your Architecture

This distinction is important.

If Spring Cloud Gateway is actually used before BPS:

Gateway responsibility

```
Routing
Authentication integration
CORS
Rate limiting
Request filtering
Observability
```

BPS responsibility

```
Payment validation
Payment business rules
Payment status
Hydra/Chase selection
External payment processing
BHS interaction
Transaction handling
Payment idempotency
```

So:

```
BASAPI / Majesco
↓
Spring Cloud Gateway
↓
BPS
↙   ↘
Hydra Chase
↓
BHS
```

That is how you should separate responsibilities when explaining the system.

20. Gateway Failure

Suppose Gateway goes down:

```
BASAPI
↓
Gateway x
```

↓
BPS

Then consumers may be unable to reach BPS through that path.

Therefore the gateway is an important infrastructure component and should be deployed with appropriate:

- redundancy
- health checks
- monitoring
- scaling
- failover

21. Security Warning

Don't treat the gateway as the **only** security boundary.

For example:

```
Gateway
↓
Authentication
↓
BPS
```

BPS should still validate important authorization/business conditions.

Also avoid logging sensitive payment data simply because the gateway can see every request.

22. Interview Questions

Basic

1. What is an API Gateway?
2. Why do we need an API Gateway?
3. What is Spring Cloud Gateway?
4. What is a route?
5. What are predicates?
6. What are filters?

Intermediate

7. API Gateway vs Load Balancer?
8. API Gateway vs Service Discovery?
9. How does Gateway work with Eureka?
10. How does Gateway perform routing?
11. How do you implement authentication?
12. How do you configure CORS?
13. How do you implement rate limiting?
14. What is path rewriting?
15. How do you configure timeouts?

3-year level

16. What happens if Gateway is down?
17. Where should business logic live?
18. Would you retry POST `/payments` at the gateway?
19. How do you prevent duplicate payment during retries?
20. How do you trace a request from BASAPI → Gateway → BPS → Hydra?
21. How do you handle downstream failures?
22. How does Gateway integrate with service discovery?
23. How would you scale the gateway?
24. How would you monitor Gateway latency/errors?

23. Very Likely Interview Scenario

Interviewer:

“BASAPI sends a payment request. Explain what happens through the API Gateway and BPS.”

A good answer, adapted to your architecture, would be:

“BASAPI sends the payment request to the configured API entry point. If Spring Cloud Gateway is used, the gateway first matches the request against its route predicates and applies the configured filters, such as security, headers, CORS or rate-limiting policies. It then routes the request to BPS, potentially using service discovery and load balancing to select a BPS instance. BPS performs the payment-specific validation and business processing, determines the appropriate downstream payment provider such as Hydra or Chase, handles the provider response, and interacts with BHS for billing history according to the system’s design.”

That keeps the **Gateway and BPS responsibilities clearly separated**.

24. One important correction to our previous topic

Now that you’ve explained your actual architecture, don’t automatically assume:

BPS → Payment Service

Your system is:

```
BASAPI / Majesco
  ↓
  BPS
  ↓
Hydra / Chase
  ↓
  BHS
```

So when we continue your interview preparation, I’ll use **BPS as the central payment service** and treat Hydra/Chase and BHS as its downstream dependencies.

Remember this distinction

API Gateway
→ Entry point + routing/policies

Service Discovery
→ Find service instances

Load Balancer
→ Choose an instance

BPS
→ Payment business logic

Hydra / Chase
→ External payment processing

BHS
→ Billing/payment history

For your preparation sequence, the natural next topic after this is **Inter-service communication: OpenFeign / RestTemplate / WebClient**, because an interviewer can move directly from “Gateway routes to BPS” to **“How does BPS communicate with Hydra, Chase and BHS?”**